Real Time DevOps Incident Story — Production Outage Example (A to Z Deep Production Understanding)

🚨 **1. REAL PRODUCTION SCENARIO — E-COMMERCE WEBSITE OUTAGE**

👉 Friday evening anuko.

👉 Big sale start ayindi — Amazon/Flipkart type e-commerce platform ki suddenly lakhs of users traffic generate chestunnaru.

👉 Users simultaneously:

- Login chestunnaru
- Products search chestunnaru
- Cart add chestunnaru
- Orders place chestunnaru
- Payments process chestunnaru

👉 Normal traffic kanna sudden ga 10x traffic increase ayindi.

📌 **Expected problem:**

High Traffic
↓
High API Requests
↓
High CPU / Memory
↓
Payment Pods Unstable
↓
5xx Errors
↓
Users Payment Failures

👉 Ikkada DevOps engineer actual production incident handling start chestadu.

🟡 **2. STEP 1 — ALERT TRIGGER AVVADAM**

👉 Production system ni Prometheus, Grafana, CloudWatch, Datadog laanti monitoring systems continuously observe chestayi.

👉 Suddenly metrics abnormal ga change ayithe alert generate avtundi.

📊 **Example Metrics**

CPU Usage → 98%
Memory Usage → 95%
API Latency → 4.8 seconds
HTTP 5xx → 18%
Pod Restarts → Increasing
Payment Success → 72%

👉 Monitoring system alert generate chestundi:

CRITICAL:
Payment Service Latency High
Error Rate > 10%

👉 PagerDuty / Slack / Email / on-call notification dwara DevOps engineer ki alert vastundi.

📌 **Explanation:** Monitoring system first problem ni detect chestundi; DevOps engineer alert receive chesi investigation start chestadu.

🔍 **Ela Check Cheyyali?**

Kubernetes environment ayithe:

```
kubectl get pods -n payment
kubectl get deployments -n payment
kubectl get nodes
```

👉 Pod restarts chudali:

```
kubectl get pods -n payment -o wide
```

👉 HPA status:

```
kubectl get hpa -n payment
```

👉 Resource usage:

```
kubectl top pods -n payment
```

kubectl top nodes

🔴 **3. STEP 2 — DEVOPS ON-CALL RESPONSE**

👉 Alert vachina ventane engineer blind ga changes cheyyadu.

👉 First **impact understand** chestadu.

Questions:

👉 Full website down aa?

👉 Only payment service affected aa?

👉 Login working unda?

👉 Product search working unda?

👉 One region lo problem aa?

👉 All regions lo problem aa?

📌 Example:

Homepage → Working ✅

Login → Working ✅

Search → Working ✅

Cart → Working ✅

Payment → Failing ❌

👉 Appudu incident scope:

Payment service outage

ani identify chestaru.

🔍 **Service Check**

kubectl get svc -n payment

kubectl get endpoints -n payment

kubectl get pods -n payment

📌 **Important:** First impact → then affected service → then root cause.

🟠 **4. STEP 3 — INITIAL QUICK CHECK**

👉 Payment pods continuously crash avtunnayi anuko.

kubectl get pods -n payment

Output:

payment-api-7d8f9 CrashLoopBackOff

payment-api-6a3k2 CrashLoopBackOff

payment-api-8h7k1 Running

👉 Pod logs check cheyyali.

kubectl logs payment-api-7d8f9 -n payment

👉 Previous crashed container logs kuda important:

kubectl logs payment-api-7d8f9 -n payment --previous

👉 Pod details:

kubectl describe pod payment-api-7d8f9 -n payment

📌 **Why --previous important?**

👉 Current container restart ayyaka previous crashed container logs useful evidence istayi.

🟣 **5. STEP 4 — CHECK WHETHER AUTOSCALING IS WORKING**

👉 Traffic increase ayindi kabatti HPA expected ga pods increase cheyyali.

🔍 **Check HPA**

kubectl get hpa -n payment

Detailed:

kubectl describe hpa payment-hpa -n payment

👉 Pods watch cheyyali:

kubectl get pods -n payment -w

👉 Example:

Before:

5 Pods

Traffic Spike:
CPU → 90%

HPA:
5 → 10 Pods
👉 But new pods create ayina immediately crash avtunnayi anuko.
📌 Appudu **scaling problem kaadu**.
👉 Application itself unhealthy ani suspect cheyyali.

🔵 6. STEP 5 — DEEP ROOT CAUSE INVESTIGATION

👉 Ippudu DevOps engineer multiple layers check chestadu.
Layer 1 — Kubernetes
kubectl describe pod <pod-name> -n payment
Layer 2 — Application Logs
kubectl logs <pod-name> -n payment
Layer 3 — Resource Usage
kubectl top pods -n payment
Layer 4 — Deployment
kubectl get deployment payment-api -n payment
Layer 5 — Recent Changes
kubectl rollout history deployment/payment-api -n payment
👉 Recent release eppudu deploy ayindo check chestaru.

🔥 7. STEP 6 — RECENT DEPLOYMENT CORRELATION

👉 Suppose:
Friday 6:00 PM
New payment version deployed

6:05 PM
Memory usage started increasing

6:10 PM
Pods started restarting

6:15 PM
Payment failures increased
📌 Timeline strong evidence istundi.
👉 Previous version healthy ga undi.
👉 New version deploy ayyaka issue start ayindi.
👉 Appudu developer + DevOps team suspect chestaru:
New release may have introduced a memory leak.

🧠 8. STEP 7 — MEMORY LEAK NI UNDERSTAND CHEYYALI

👉 Memory leak ante application unnecessary memory continuously hold chestundi.
Example:

Time	Memory
6:00 PM	40%
6:05 PM	55%
6:10 PM	72%
6:15 PM	88%
6:20 PM	97%
6:21 PM	OOMKilled

👉 Kubernetes lo pod status check chesthe:
kubectl describe pod <pod-name> -n payment
👉 Events lo OOMKilled / memory-related termination evidence kanipinchachu.
📌 **Meaning:** Container available memory limit exceed chesi terminate ayindi.

🚨 **9. STEP 8 — WAR ROOM START**

👉 Incident customer-impacting and critical ayithe war room create chestaru.

Team Members:

👷 DevOps Engineer

👷 Application Developer

🧪 QA Engineer

👨‍💼 Team Lead / Incident Commander

🔐 Security Engineer — required ayithe

👉 Main questions:

“What changed?”

“What is the customer impact?”

“Can we rollback safely?”

“Is there a permanent fix ready?”

📌 **Important:** Production incident time lo blame game undadu.

👉 Main goal:

Restore service first. Investigate deeply after stabilization.

⚡ **10. STEP 9 — DECISION: ROLLBACK OR HOTFIX?**

👉 Team two options consider chestundi.

Option 1 — Hotfix

👉 New code lo bug immediately fix chesi release cheyyadam.

Option 2 — Rollback

👉 Previous stable version ki return avvadam.

👉 Production outage already severe ayithe stable version rollback usually faster recovery option avvachu, provided rollback safe and tested.

📌 Example:

Current:

payment:v2 ❌

Previous:

payment:v1 ✅

👉 Decision:

Rollback v2 → v1

🔄 **11. STEP 10 — KUBERNETES ROLLBACK**

👉 First rollout history:

```
kubectl rollout history deployment/payment-api -n payment
```

👉 Previous revision ki rollback:

```
kubectl rollout undo deployment/payment-api -n payment
```

👉 Rollout status:

```
kubectl rollout status deployment/payment-api -n payment
```

👉 Pods:

```
kubectl get pods -n payment
```

👉 Logs:

```
kubectl logs <new-pod-name> -n payment
```

📌 **Goal:** Old stable application version restore cheyyadam.

🟢 **12. STEP 11 — TRAFFIC NORMALIZATION**

👉 Old version pods healthy ga run avuthunnayi.

👉 Kubernetes Service healthy endpoints ki traffic route chestundi.

👉 Load balancer healthy targets ni continue chestundi.

Check:

```
kubectl get endpoints -n payment
kubectl get pods -n payment
```

👉 Application health endpoint test:
curl -I https://example.com/health
👉 Actual payment flow ni controlled test request tho verify chestaru.
📌 **Important:** Pod Running undadam alone sufficient kaadu.
👉 Application actually requests process chestunda ani verify cheyyali.

📊 **13. STEP 12 — RECOVERY MONITORING**
👉 Rollback ayyaka immediately incident close cheyyaru.
👉 At least key metrics observe chestaru.
Before:
CPU → 98%
Memory → 95%
Latency → 4.8 sec
Errors → 18%
After:
CPU → 45%
Memory → 52%
Latency → 180 ms
Errors → 0.5%
👉 Grafana / CloudWatch dashboards observe chestaru.
👉 Payment success rate normal ayinda check chestaru.
📌 **Recovery means:** Infrastructure healthy + application healthy + user transactions successful.

🛡️ **14. STEP 13 — INCIDENT STABILIZATION**
👉 System recover ayina tarvata temporary protections apply cheyyachu.
👉 Example:

- Deployment freeze
- Problematic version stop
- Traffic limits review
- HPA settings verify
- Resource limits review
- Error alerts maintain
- Extra monitoring enable

📌 **Goal:** Same incident immediately repeat kakunda system stable ga maintain cheyyadam.

📝 **15. STEP 14 — RCA (ROOT CAUSE ANALYSIS)**
👉 Next day incident team complete RCA prepare chestundi.
RCA Questions
What happened?
👉 Payment service became unstable.
When?
👉 After new deployment.
What was affected?
👉 Payment transactions.
Root Cause?
👉 Application memory leak.
Why wasn't it detected before production?
👉 Load testing / memory monitoring gap.
How was service restored?
👉 Rollback to previous stable version.

🔧 **16. STEP 15 — PERMANENT FIX**
👉 Rollback temporary recovery.
👉 Permanent solution application code lo memory leak fix cheyyadam.

👉 Developer bug fix chestadu.
👉 QA testing chestundi.
👉 Load testing chestaru.
👉 CI/CD pipeline lo automated tests add chestaru.
👉 Then controlled deployment.
📌 **Real DevOps thinking:**
Rollback = Restore
Code Fix = Permanent Solution

🩹 **17. STEP 16 — HOW TO PREVENT SAME INCIDENT?**
👉 Same problem future lo repeat kakunda multiple layers improve chestaru.
Application Level
👉 Memory leak fix.
CI/CD Level
👉 Automated tests.
👉 Static analysis.
👉 Security scanning.
Performance Level
👉 Load testing.
👉 Stress testing.
Kubernetes Level
👉 Correct resource requests/limits.

```
resources:
 requests:
  memory: "256Mi"
  cpu: "250m"
 limits:
  memory: "512Mi"
  cpu: "500m"
```

Monitoring Level
👉 Memory trend alerts.
👉 Restart count alerts.
👉 OOMKilled alerts.
Deployment Level
👉 Canary / blue-green deployment consider cheyyachu.
📌 **Result:** Next release direct ga millions of users meeda impact kakunda small traffic tho first validate cheyyachu.

🔍 **18. REAL DEVOPS COMMAND FLOW — COMPLETE**
👉 Production incident start ayinappudu practical ga commands ila flow avvachu:
1️⃣ **Check Nodes**

```
kubectl get nodes
```

2️⃣ **Check Pods**

```
kubectl get pods -A
```

3️⃣ **Check Payment Pods**

```
kubectl get pods -n payment
```

4️⃣ **Check Logs**

```
kubectl logs <pod-name> -n payment
```

5️⃣ **Previous Crash Logs**

```
kubectl logs <pod-name> -n payment --previous
```

6️⃣ **Pod Details**

```
kubectl describe pod <pod-name> -n payment
```

7️⃣ **Resource Usage**

```
kubectl top pods -n payment
```

8️⃣ **HPA**

```
kubectl get hpa -n payment
```

9️⃣ **Deployment**
kubectl get deployment -n payment
🔟 **Deployment History**
kubectl rollout history deployment/payment-api -n payment
1️⃣1️⃣ **Rollback**
kubectl rollout undo deployment/payment-api -n payment
1️⃣2️⃣ **Verify Rollout**
kubectl rollout status deployment/payment-api -n payment
1️⃣3️⃣ **Verify Pods**
kubectl get pods -n payment
1️⃣4️⃣ **Verify Service**
kubectl get svc -n payment
1️⃣5️⃣ **Verify Endpoints**
kubectl get endpoints -n payment
📌 **Command flow:**
Get → Describe → Logs → Metrics → History → Fix → Verify

🧠 **19. IF THE FIX DOESN'T WORK — NEXT LEVEL THINKING**
👉 Rollback chesina tarvata kuda issue continue ayithe immediately "application bug" ani assume cheyyakudadhu.
👉 Other layers check cheyyali.
Database
Connection pool
Slow queries
CPU
Storage
Locks
Network
DNS
Security Groups
Network Policies
Load Balancer
Service endpoints
Infrastructure
Node capacity
Disk
Memory
CPU
Availability Zone
External Dependency
Payment gateway
Third-party API
Authentication provider
📌 **Real production troubleshooting:** One layer meeda fixation kakunda **layer-by-layer isolation** cheyyali.

🧠 **20. COMPLETE INCIDENT FLOW**
User Traffic Spike
↓
Monitoring Detects Problem
↓
Alert
↓
DevOps On-Call
↓
Impact Analysis
↓
Check Kubernetes
↓

Check Pods
↓
Check Logs
↓
Check Metrics
↓
Check Recent Deployment
↓
Identify Memory Leak
↓
War Room
↓
Rollback
↓
Pods Recover
↓
Traffic Normal
↓
Monitor
↓
RCA
↓
Permanent Code Fix
↓
Testing
↓
Safer Deployment
↓
Prevention

🔥 **21. REAL LESSON FROM THIS INCIDENT**

👉 DevOps engineer job ante **“pod restart cheyyadam”** matrame kaadu.

👉 Real responsibility:

Detect → Understand Impact → Troubleshoot → Restore → Verify → Analyze → Prevent

👉 Production incident lo first question:

“Who caused it?” ❌

👉 Correct question:

“How quickly can we restore the service safely?” ✅

👉 Incident recover ayyaka:

“Why did this happen, and how do we make sure it doesn't happen again?”

🔥 **FINAL DEVOPS TRUTH:**

👉 **“A strong DevOps engineer is not just someone who fixes a production outage; they build automation, monitoring, deployment safety, and architecture so that the same outage becomes less likely—and when failure happens, recovery is fast and controlled.”**